Teacher's Methodology Guide

Lecture 3: Informed Search

How to Solve Exercises the Way Your Teacher Expects

Extracted from Exercise Sheet 3 Solutions

Contents

1 Introduction: Understanding Your Teacher's Approach **2**
- 1.1 What Makes This Guide Different . . . 2
- 1.2 Structure of This Guide . . . 2

2 Part 1: Heuristic Properties **3**
- 2.1 Exercise Type: Proving Admissibility . . . 3
- 2.2 Exercise Type: Proving Consistency . . . 4
- 2.3 Exercise Type: Designing Heuristics . . . 5

3 Part 2: Greedy Best-First Search (GBFS) **6**
- 3.1 Table Format Requirements . . . 6
- 3.2 Notation Rules . . . 7
- 3.3 Tie-Breaking Rules . . . 7
- 3.4 Drawing Search Trees . . . 7

4 Part 3: A* Search **9**
- 4.1 Table Format Requirements . . . 9
- 4.2 Three Cases for Duplicate Handling . . . 9
- 4.3 Explaining Optimality . . . 10

5 Part 4: Common Patterns Across All Exercises **12**
- 5.1 General Writing Style . . . 12
- 5.2 Proof Structure . . . 12
- 5.3 Common Mistakes to Avoid . . . 13

6 Part 5: Exercise-Specific Checklists **15**
- 6.1 Heuristic Property Proofs . . . 15
- 6.2 GBFS Execution . . . 15
- 6.3 A* Execution . . . 16
- 6.4 Search Tree Drawing . . . 16

7 Part 6: Quick Reference **17**
- 7.1 Notation Quick Reference . . . 17
- 7.2 Algorithm Comparison Quick Table . . . 17
- 7.3 Sentence Templates . . . 18

8 Conclusion: Mastering the Teacher's Style **19**
- 8.1 The Key Insights . . . 19
- 8.2 How to Use This Guide . . . 19
- 8.3 Final Checklist . . . 20

1 Introduction: Understanding Your Teacher's Approach

This guide extracts the **exact methodology** your teacher uses when solving Lecture 3 exercises. By following these patterns, you'll match the expected solution format and reasoning style.

1.1 What Makes This Guide Different

Three Levels of Understanding

Level 1: Algorithm Knowledge

- Know what GBFS and A* do
- Understand admissibility and consistency

Level 2: Problem-Solving Ability

- Can execute algorithms
- Can prove heuristic properties

Level 3: Teacher's Methodology ← This Guide

- Know the *exact format* expected
- Use the *exact reasoning patterns*
- Structure answers *exactly as teacher does*

1.2 Structure of This Guide

Each section covers one exercise type with:

1. **Teacher's Template** - Exact format to use
2. **Annotated Example** - From actual solutions
3. **Common Mistakes** - What students get wrong
4. **Grading Criteria** - What teacher looks for

2 Part 1: Heuristic Properties

2.1 Exercise Type: Proving Admissibility

Teacher's Methodology for Admissibility Proofs

Structure Required:

1. **Answer Yes/No first** - Be explicit
2. **One-sentence justification** - Teacher asks for 1-2 sentences
3. **Optional**: Add "(alternatively: ...)" for second reasoning

Acceptable Reasoning Patterns:

- Pattern 1: "It is [name of distance metric]"
- Pattern 2: "Because [physical/geometric argument]"
- Pattern 3: "Consistency implies admissibility which was shown in [X]"

Exercise 1.1 Part 2: Teacher's Solution

Question: Is $h((u, v)) = |u - x| + |v - y|$ an admissible heuristic?
Teacher's Answer:

> Yes, this is an admissible heuristic; it is the Manhattan distance metric (alternatively: consistency implies admissibility which was shown in 1.).

Analysis of Format:

- **Yes** - Direct answer
- **; it is the Manhattan distance metric** - Primary justification
- **(alternatively: ...)** - Secondary justification
- **Length: 1 sentence** - Meets 1-2 sentence requirement

Template for Admissibility Proof

Format:

```
[Yes/No], [justification reason].
```

Common Justifications:

- "it is the [distance metric name]"
- "removing links may induce detours, which require more steps"
- "they need to [action], and so they need to move at least that amount"
- "consistency implies admissibility"

Don't Write:

- Long proofs (teacher wants 1-2 sentences!)
- Formal mathematical induction
- Multiple paragraphs

2.2 Exercise Type: Proving Consistency

Teacher's Methodology for Consistency Proofs

To Prove Consistent:

1. State: "Yes" or "Yes, this is consistent"
2. Pick arbitrary transition $s \xrightarrow{a} s'$
3. Show: $h(s) - h(s') \leq \text{cost}(a)$ for all cases
4. Conclude with geometric argument if applicable

To Prove NOT Consistent (Counterexample):

1. State: "No" or "No, this is not consistent"
2. Write: "Consider state $s =$ [concrete state]"
3. Write: "and one of its successors $s' =$ [concrete successor]"
4. Calculate: $h(s) =$ [value], $h(s') =$ [value], cost $=$ [value]
5. Show: $h(s) - h(s') =$ [difference] $>$ cost
6. Conclude: "Therefore we have that $h(s) - h(s') = X$, but for a consistent h we would need to have $h(s) - h(s') \leq Y$."

Exercise 1.2: Teacher's Consistency Proof (Negative)

Question: Is the heuristic consistent?
Teacher's Answer:

> No.
>
> Consider state $s = (a = D, e = D, f = C)$ and one of its successors $s' = (a = A, e = D, f = C)$. We have that $h(s) = 4$, $h(s') = 1$ and cost of the transition from s to s' is 1. Therefore we have that $h(s) - h(s') = 3$, but for a consistent h we would need to have $h(s) - h(s') \leq 1$.

Key Elements:

1. **Clear "No"** at the start
2. **Concrete states** - Not abstract s and s'
3. **All three values** - $h(s)$, $h(s')$, cost shown
4. **Exact calculation** - Shows $h(s) - h(s') = 3$
5. **Comparison to requirement** - "would need to have ≤ 1"

Template for Consistency Counterexample

Format:

```
No.

Consider state s = (concrete state description) and one of its
successors s' = (concrete successor). We have that h(s) = [X],
```

```
h(s') = [Y] and cost of the transition from s to s' is [Z].
Therefore we have that h(s) - h(s') = [X-Y], but for a consistent h
we would need to have h(s) - h(s')  [Z].
```

Critical Requirements:

- Use **concrete examples**, not variables
- Show **all calculations** explicitly
- Use exact phrasing: "but for a consistent h we would need..."
- **One counterexample is sufficient**

2.3 Exercise Type: Designing Heuristics

Teacher's Methodology for Heuristic Design

Structure:

1. **State your heuristic** - Clear definition
2. **Argue admissibility** - Why it never overestimates
3. **Argue consistency** - Show it holds or give counterexample
4. **Optional**: Discuss trade-offs or alternatives

Teacher's Reasoning Pattern:

- "It is admissible because they need to [action], and so they need to [lower bound]"
- "Although it can still underestimate a lot"
- "[Alternative heuristic] would give us both admissible and consistent"

Exercise 1.3: Teacher's Heuristic Design

Question: Design an admissible and consistent heuristic for Elevator Scheduling.
Teacher's Solution Structure:

1. **Define notation:** "Let d_i denote..."
2. **Define function:** "Let D be the function defined as..."
3. **State heuristic:** "we define $h(...) = \max_{i \in \{...\}} D(i)$"
4. **Explain:** "In other words, we compute..."

Key Pattern: Teacher uses **mathematical notation** first, then explains in words.

3 Part 2: Greedy Best-First Search (GBFS)

3.1 Table Format Requirements

Teacher's GBFS Table Format

Exact Columns Required:

expanded node	nodes that might get added	open	closed

Important Details:

- Column 1: Single node (state representation)
- Column 2: Set notation with braces: {node1, node2, ...}
- Column 3: List notation with brackets: [(node1, h), (node2, h), ...]
- Column 4: Set notation with braces: {state1, state2, ...}

Ordering in Open:

- Always order by h-value (lowest first)
- Show h-values explicitly when needed
- Apply tie-breaking rule consistently

Exercise 2.1: Teacher's GBFS Table

From Exercise Sheet Solution:

expanded node	nodes that might get added	open	closed
		[(1,1)]	{}
(1,1)	{(2,1), (1,2)}	[(2,1), (1,2)]	{(1,1)}
(2,1)	{(3,1), (2,0), (1,1)}	[(3,1), (1,2), (2,0)]	{(1,1), (2,1)}

Key Observations:

1. **Initial row**: Empty expanded, open has initial state, closed empty
2. **Set notation**: "nodes that might get added" uses {}
3. **List notation**: "open" uses []
4. **Duplicate handling**: (1,1) in "nodes that might get added" but NOT added to open (it's in closed)

3.2 Notation Rules

Teacher's Notation for States

Grid Problems: Use coordinates (x, y)

- Example: (1,1), (2,3), (4,1)

Meeting Point Problems: Use location letters

- Full notation: $(a = A, e = E, f = F)$
- Compressed notation: AEF (when clear from context)

State Space Graphs: Use node letters

- Example: A, B, C

Consistency: Teacher uses both notations interchangeably but compressed is preferred in tables.

3.3 Tie-Breaking Rules

Teacher's Tie-Breaking Methodology

When Problem Specifies Lexicographic Order:

1. Compare first component (lowest wins)
2. If tied, compare second component
3. Continue until tie is broken

Example from Exercise 2.1:

> "break ties by lower Y value and if that is the same, lower X value"

This means: $(3, 0)$ before $(2, 1)$ before $(4, 1)$

- $(3, 0)$: Y=0 (lowest Y)
- $(2, 1)$: Y=1, X=2
- $(4, 1)$: Y=1, X=4 (higher X than $(2, 1)$)

When Problem Doesn't Specify:

- Assume lexicographic ordering
- Or state your assumption clearly

3.4 Drawing Search Trees

Teacher's Search Tree Format

Required Elements:

1. **Node labels**: State name with h-value underneath
2. **Crossed nodes**: X through states not added (duplicate elimination)
3. **Circled goal**: Circle around goal state when found

4. **Expansion order**: Numbers (1), (2), (3)... to show order

Layout:

- Root at top
- Children below parents
- Left-to-right ordering by action order

Exercise 2.2.b: Teacher's Search Tree

From Image Analysis:
The teacher draws:

- **State labels**: DEF, BEF, ACF, ADF, AFF, AED, AEE
- **h-values below**: 6, 10, 12, 9, 4, 6, 6
- **Circled goal**: DDD with h=0 circled
- **Crossed states**: Some child nodes have X through them

Format Pattern:

```
        AEF
       h = 8
      /  |  \
  DEF  BEF  ACF  ...
   6    10   12
```

4 Part 3: A* Search

4.1 Table Format Requirements

Teacher's A* Table Format

Exact Columns Required:

Expand	g	h	f	Open	Closed

OR (more compact):

expanded node	nodes added	open	closed

With notation: State(g/h) or State: g/h
Critical Difference from GBFS:

- Must show both g and h values
- Open list ordered by $f = g + h$ (not just h!)
- Must handle cheaper path updates

Exercise 2.3: Teacher's A* Table Structure

Teacher uses annotation format:

```
A (1)
g = 0, h = 3

C (2)
g = 5, h = 2

G (3)
g = 6, h = 1
```

Where:

- (1), (2), (3) = expansion order
- g = path cost from start
- h = heuristic value
- $f = g + h$ (computed but sometimes implicit)

4.2 Three Cases for Duplicate Handling

Teacher's Three-Case Logic for A*

When generating successor n':
Case 1: New State

- Condition: $n'.State \notin closed$ **and** $n'.State \notin \text{States}(open)$
- Action: INSERT n' into open
- Teacher writes: "Insert" or just adds to open list

Case 2: State in Open with Higher g

- Condition: $\exists n'' \in open$ with same state and $g(n') < g(n'')$
- Action: REPLACE n'' with n' in open
- Teacher writes: "Update" or "Replace" or shows updated values

Case 3: State in Closed with Higher g

- Condition: $\exists n'' \in closed$ with same state and $g(n') < g(n'')$
- Action: REMOVE from closed, INSERT n' into open (reopening)
- Teacher writes: "Reopen" or moves state back to open

Important: If $g(n') \geq g(n'')$, do nothing (the old path is better)!

Exercise 2.3: Teacher Handles Updates

From Solution Diagram:
Teacher shows crossed-out nodes when:

- Node would be added but state is in closed (X through it)
- Node would be added but worse path exists (X through it)

Teacher shows updated nodes when:

- Better path found to state in open
- Explicitly notes the update

4.3 Explaining Optimality

Teacher's Optimality Explanation Format

Required Components:

1. State that solution is optimal (or not)
2. Explain *why* using A* properties
3. Reference admissibility of heuristic if relevant

Teacher's Phrasing Patterns:

- "The solution is A, C, H, I. A* is guaranteed to return an optimal solution if..."
- "Yes, because the heuristic is admissible even though it is not consistent."
- "The heuristic is admissible because..."

Exercise 2.3.b: Teacher's Optimality Explanation

Question: Explain why the found solution is optimal.
Teacher's Answer:

> The solution is A, C, H, I. A* is guaranteed to return an optimal solution if the heuristic that is being used is admissible. The heuristic is admissible because we do not have any zero-cost edges and thus the length of the shortest path is always

lower-bound on the cost of that path.

Structure Analysis:

1. **State solution**: "The solution is A, C, H, I"
2. **General principle**: "A* is guaranteed to return an optimal solution if..."
3. **Verify condition**: "The heuristic is admissible because..."
4. **Problem-specific reasoning**: "we do not have any zero-cost edges..."

5 Part 4: Common Patterns Across All Exercises

5.1 General Writing Style

Teacher's Writing Style Characteristics

Sentence Structure:

- Direct and concise
- Uses semicolons to connect related thoughts
- Parenthetical alternatives: "(alternatively: ...)"
- Mathematical precision when needed

Answer Format:

- Always answer the question directly first (Yes/No, value, etc.)
- Then provide justification
- Keep justifications brief (1-2 sentences when specified)

Mathematical Notation:

- Uses set notation consistently: {} for sets, [] for lists
- Subscripts for variables: s', n', n''
- Clear function notation: $h(s)$, $g(n)$, $f(n)$

5.2 Proof Structure

Teacher's Proof Patterns

For Showing Property Holds:

1. State the property formally
2. Give intuitive explanation
3. Optional: Add formal argument if needed
4. Conclude definitively

For Showing Property Fails:

1. State "No" or "It does not hold"
2. Provide concrete counterexample
3. Show explicit calculations
4. Point out the violation: "but we would need..."

Avoid:

- Informal hand-waving
- Overly lengthy formal proofs
- Missing the direct answer

5.3 Common Mistakes to Avoid

Top 10 Student Mistakes

1. **Wrong Notation**
   - Using $h(n)$ instead of $h(n.State)$ in formal contexts
   - Confusing states and nodes
2. **Table Format Errors**
   - Wrong column headers
   - Missing g/h/f values for A*
   - Incorrect ordering in open list
3. **Tie-Breaking Mistakes**
   - Not following specified tie-breaking rule
   - Applying rule inconsistently
4. **Duplicate Handling**
   - Adding states that are already in closed (GBFS)
   - Not updating cheaper paths (A*)
   - Forgetting Case 3 (reopening) for A*
5. **Admissibility vs Consistency**

- Claiming admissibility implies optimality for GBFS (it doesn't!)
- Not distinguishing between the two properties

6. Proof Format

- Too verbose (teacher wants 1-2 sentences!)
- Not starting with Yes/No
- Abstract counterexample instead of concrete

7. Search Tree Errors

- Not marking duplicate elimination
- Not showing h-values
- Not circling goal

8. Ordering Mistakes

- GBFS ordered by f instead of h
- A* ordered by h instead of f
- Not computing f = g + h correctly

9. Goal Test Timing

- Testing goal when generating (wrong!)
- Should test when expanding (popping from open)

10. Solution Quality

- Claiming GBFS finds optimal solution
- Not explaining why A* solution is optimal

6 Part 5: Exercise-Specific Checklists

6.1 Heuristic Property Proofs

Checklist for Admissibility

Before submitting, verify:

- ☐ Answered Yes/No explicitly
- ☐ Justification is 1-2 sentences (as required)
- ☐ Used acceptable reasoning pattern
- ☐ Ended with period
- ☐ Mentioned metric name if applicable

Checklist for Consistency

Before submitting, verify:

- ☐ Answered Yes/No explicitly
- ☐ If No: Provided concrete counterexample
- ☐ If No: Showed all three values ($h(s)$, $h(s')$, cost)
- ☐ If No: Calculated $h(s) - h(s')$ explicitly
- ☐ If No: Used phrasing "but for a consistent h we would need..."
- ☐ If Yes: Showed property holds for general case

6.2 GBFS Execution

Checklist for GBFS

Before submitting, verify:

- ☐ Table has correct column headers
- ☐ Initial row shows initial state in open, empty closed
- ☐ "nodes that might get added" uses set notation {}
- ☐ "open" uses list notation [] with ordering
- ☐ "closed" uses set notation {}
- ☐ Open list ordered by h-value (lowest first)
- ☐ Tie-breaking rule applied consistently
- ☐ Duplicate elimination correct (skip if in open or closed)
- ☐ Goal marked when found
- ☐ Correct number of iterations as specified

6.3 A* Execution

Checklist for A*

Before submitting, verify:

- ☐ Table shows g, h, and f values (or g/h notation)
- ☐ Open list ordered by f-value (not h!)
- ☐ f = g + h calculated correctly for each node
- ☐ Case 1 (new state): Added to open
- ☐ Case 2 (in open, cheaper): Updated
- ☐ Case 3 (in closed, cheaper): Reopened
- ☐ States with higher/equal g not updated
- ☐ Goal marked when found
- ☐ Optimality explained with reference to admissibility

6.4 Search Tree Drawing

Checklist for Search Trees

Before submitting, verify:

- ☐ Root node at top
- ☐ Each node labeled with state
- ☐ h-value shown below/beside each node
- ☐ For A*: g-value also shown
- ☐ Expansion order marked (1), (2), (3)...
- ☐ Duplicate nodes crossed out (X through them)
- ☐ Goal node circled
- ☐ Tree structure clear (parent-child relationships obvious)

7 Part 6: Quick Reference

7.1 Notation Quick Reference

Complete Notation Guide

States vs Nodes:

- s, s', s'' - states in state space
- n, n', n'' - search nodes
- $n.State$ - state associated with node n

Values:

- $h(s)$ or $h(n.State)$ - heuristic estimate
- $g(n)$ - path cost from start to n
- $f(n) = g(n) + h(n.State)$ - total estimated cost
- $h^*(s)$ - true optimal cost from s to goal

Data Structures:

- *open* - priority queue of nodes (list notation [])
- *closed* - set of expanded states (set notation {})
- States(*open*) - set of states with nodes in open

Node Components:

- $n.State$ - the state
- $n.Pred$ - predecessor node
- $n.Action$ - action from predecessor
- $g(n)$ - cost from start

Actions:

- a - an action
- $s \xrightarrow{a} s'$ - transition notation
- cost(a) or $c(a)$ - action cost

7.2 Algorithm Comparison Quick Table

Property	UCS	GBFS	A*
Orders by	$g(n)$	$h(n.State)$	$g(n) + h(n.State)$
Complete	Yes	Yes	Yes
Optimal	Yes	NO	Yes (if h admissible)
Uses heuristic	No	Yes	Yes
Cheaper path check	Yes	NO	Yes
Reopening (Case 3)	Yes	NO	Yes

7.3 Sentence Templates

Ready-to-Use Sentence Templates

Admissibility:

- "Yes, this is an admissible heuristic; it is the [metric name]."
- "Yes; [reason why it never overestimates]."
- "No; [counterexample showing overestimation]."

Consistency:

- "No. Consider state s = [state] and one of its successors s' = [state]. We have that $h(s) = X$, $h(s') = Y$ and cost of the transition from s to s' is Z. Therefore we have that $h(s) - h(s') = W$, but for a consistent h we would need to have $h(s) - h(s') \leq Z$."

Optimality:

- "The solution is [path]. A* is guaranteed to return an optimal solution if the heuristic is admissible. The heuristic is admissible because [reason]."
- "Yes, because the heuristic is admissible even though it is not consistent."

Problem-Specific Reasoning:

- "because they need to [action], and so they need to move at least that amount"
- "because we do not have any zero-cost edges and thus..."
- "removing links may induce detours, which require more steps"

8 Conclusion: Mastering the Teacher's Style

8.1 The Key Insights

What Separates Good from Great Answers

Good Answer:

- Correct algorithm execution
- Right final answer
- Shows understanding

Great Answer (What Teacher Wants):

- **Exact format** matching teacher's style
- **Precise notation** (states vs nodes, h(n.State), etc.)
- **Structured reasoning** following expected patterns
- **Appropriate length** (1-2 sentences when asked)
- **Clear presentation** (tables, trees formatted correctly)

8.2 How to Use This Guide

1. **Before Solving:**
   - Identify exercise type
   - Find relevant section in this guide
   - Review template and checklist
2. **While Solving:**
   - Follow teacher's format exactly
   - Use notation guide for correct symbols
   - Match sentence structure patterns
3. **After Solving:**
   - Use checklist to verify completeness
   - Check format matches examples
   - Verify length requirements
4. **During Review:**
   - Compare your solution to teacher's examples
   - Note any formatting differences
   - Practice matching the style exactly

8.3 Final Checklist

Before Submitting ANY Exercise

Ask yourself:

1. Does my notation match the teacher's exactly?
2. Have I answered Yes/No explicitly when required?
3. Are my justifications the right length (1-2 sentences)?
4. Do my tables have the correct column headers?
5. Is my open list ordered by the correct criterion?
6. Have I handled all three cases for A*?
7. Are my counterexamples concrete (not abstract)?
8. Have I marked duplicate elimination in search trees?
9. Does my explanation reference the right properties?
10. Would the teacher recognize my answer as matching their style?

If you can answer YES to all of these, you're ready!

Remember:

The teacher has a specific style.
Match it exactly, and you'll excel.

This guide gives you the patterns.
Practice makes perfect!